

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Code of Conduct:

I K Uday Kiran Reddy (192425211) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: k Uday Kiran

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Question 1 – RL Framework and Agent–Environment Interaction

Reinforcement Learning (RL) is a learning framework in which an **agent learns to make decisions by interacting with an environment**. At each time step, the agent observes the current **state**, selects an **action**, receives a **reward**, and moves to a new state.

The interaction can be represented as:

State → Action → Reward → Next State

- **Agent:** The learner or decision-maker.
- **Environment:** The external system with which the agent interacts.
- **State (S):** Represents the current situation of the environment.
- **Action (A):** A decision taken by the agent.
- **Reward (R):** Numerical feedback indicating how good or bad the action was.

The objective of the agent is to maximize the **cumulative reward**:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where γ is the discount factor.

For example, in a warehouse robot, the state represents the robot's location, the action may be moving left/right, and the reward may be positive for reaching the destination and negative for collisions.

Thus, cumulative reward influences decision-making because the agent does not only consider the immediate reward but also the **long-term consequences of its actions**.

Analysis:

An effective RL framework continuously learns from feedback. Therefore, states, actions, and rewards form the foundation for learning an optimal decision-making strategy.

Question 2 – Markov Decision Process

A **Markov Decision Process (MDP)** provides a mathematical framework for modeling sequential decision-making problems.

An MDP can be represented as:

$$MDP = (S, A, P, R, \gamma)$$

where:

- **S**: Set of states.
- **A**: Set of possible actions.
- **$P(s'|s, a)$** : Transition probability from state (s) to state (s') after action (a) .
- **$R(s, a)$** : Reward obtained after taking action (a) in state (s) .
- **γ** : Discount factor that determines the importance of future rewards.

Example

Consider a robot navigating a warehouse.

Component	Example
State	Robot's current position
Action	Move left, right, forward, backward
Transition Probability	Probability of reaching the intended position
Reward	+10 for reaching destination, -10 for collision
Discount Factor	0.9

The **Markov property** means that the next state depends primarily on the current state and action rather than the complete history.

Analysis:

MDPs allow complex decision-making problems to be represented systematically. By defining states, actions, transitions, rewards, and discounting, an RL agent can learn an effective policy.

Question 3 – Policy and Value Functions

A **policy** defines the strategy used by an RL agent to select actions.

$$\pi(a|s) = P(A_t = a | S_t = s)$$

A policy may be deterministic or stochastic.

State-Value Function

The state-value function represents the expected long-term reward from a state while following a particular policy.

$$V^\pi(s) = E_\pi[G_t | S_t = s]$$

Action-Value Function

The action-value function represents the expected long-term reward for taking action a in state s and subsequently following the policy.

$$Q^\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$$

The Bellman equation for the state-value function is:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

The agent can compare different actions using the Q-value and select an action with a higher expected long-term benefit.

Example:

If a robot can move left or right and:

$$Q(s, \text{left}) = 5 \quad Q(s, \text{right}) = 10$$

the agent would prefer **right**, assuming it follows a greedy strategy.

Analysis:

Policy determines **what the agent does**, while value functions estimate **how beneficial those decisions are**. Together, they enable the agent to improve its decision-making.

Question 4 – Exploration–Exploitation Trade-Off

RL agents face a trade-off between:

- **Exploration:** Trying new actions to discover potentially better strategies.
- **Exploitation:** Selecting the best-known action based on current knowledge.

ϵ -Greedy Strategy

In ϵ -greedy learning:

- With probability ϵ , the agent chooses a random action.
- With probability $1 - \epsilon$, it chooses the action with the highest estimated value.

For example, if:

$$\epsilon = 0.1$$

the agent explores approximately 10% of the time and exploits approximately 90% of the time.

Softmax Strategy

Softmax assigns probabilities to actions according to their estimated values:

$$P(a) = \frac{e^{Q(s,a)}}{\sum_b e^{Q(s,b)}}$$

where τ is the temperature parameter.

A higher temperature encourages exploration, while a lower temperature favors the action with the highest value.

Comparison

ϵ -Greedy	Softmax
Simple to implement	More gradual action selection
Random exploration	Value-based exploration
Easy to understand	Uses probability distribution
May select poor actions randomly	Better differentiates action values

Analysis:

Exploration is necessary to discover better strategies, while exploitation uses existing knowledge. Proper balancing improves learning efficiency.

Question 5 – Reward Shaping

Reward shaping is the process of designing additional rewards or penalties to guide an RL agent toward desirable behavior.

Instead of providing a reward only when the final goal is achieved, intermediate rewards can be provided.

Example

Consider a robot navigating toward a destination.

- Reaching destination: **+100**
- Moving closer: **+5**
- Moving farther away: **-5**
- Collision: **-50**

Without reward shaping, the robot may receive very little feedback during learning.

With reward shaping, the agent receives continuous guidance toward the goal.

A simple shaped reward can be expressed as:

$$R' = R + F(s,s')$$

where $F(s,s')$ is an additional shaping reward.

Impact:

1. Speeds up learning.
2. Provides more informative feedback.
3. Encourages desirable behavior.
4. Reduces unnecessary exploration.
5. Helps avoid poor policies.

Analysis:

Poorly designed rewards can create unintended behavior. Therefore, reward shaping must encourage behavior that is consistent with the actual objective.

Question 6 – RL in Real-World Robotics

Applying RL to real-world robotics is challenging because robots operate in complex and uncertain environments.

Major Challenges

1. Safety

An incorrect action may damage the robot or harm people.

Example: A robotic arm may collide with an object.

2. Sample Efficiency

RL may require a large number of interactions to learn a good policy. Real-world interactions are expensive and time-consuming.

3. Environment Variability

Real environments may change because of:

- Lighting
- Obstacles
- Sensor noise
- Moving objects
- Different operating conditions

4. Sensor Uncertainty

Sensors may provide incomplete or inaccurate information.

5. Computational Requirements

Learning and decision-making may need to occur quickly.

Analysis

Simulation can be used to train robots before deployment. Safety constraints and controlled exploration can reduce risks.

Thus, an effective robotics RL system should combine **safe learning, efficient training, robust sensing, and adaptation to environmental changes.**

Question 7 – Discount Factor γ

The **discount factor** γ determines how much importance the agent gives to future rewards.

Its value normally lies between:

$$[0 \leq \gamma \leq 1]$$

The return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Effect of γ

Low γ :

The agent focuses more on immediate rewards.

High γ :

The agent gives greater importance to future rewards.

Example

Suppose an agent has two choices:

- Action A $\rightarrow$ immediate reward = 5
- Action B $\rightarrow$ future reward = 20

With a low discount factor, the agent may prefer A.

With a high discount factor, the agent may prefer B because future rewards have greater importance.

Episodic Tasks

In episodic tasks such as games or maze navigation, γ determines whether the agent focuses on immediate progress or the total reward accumulated before the episode ends.

Analysis:

Choosing γ appropriately is important. A value that is too low can result in short-sighted decisions, while a high value encourages long-term planning.

Question 8 – Model-Free vs Model-Based RL

Model-Free RL

Model-free RL learns directly from interactions without explicitly learning the environment's transition model.

Examples:

- Q-learning
- SARSA

Advantages:

- Simple environment representation.
- Useful when the environment model is unknown.
- Can work effectively with direct experience.

Limitations:

- May require many interactions.
- Learning can be slow in complex environments.

Model-Based RL

Model-based RL uses or learns a model of the environment.

The model estimates:

$$P(s'|s,a)$$

and rewards to predict future outcomes.

Advantages:

- Can plan before taking actions.
- More sample efficient in some situations.
- Useful when environment dynamics can be modeled.

Limitations:

- Requires an accurate model.
- Model errors can lead to poor decisions.
- More computationally complex.

Comparison

Feature	Model-Free	Model-Based
Environment model	Not required	Required/learned
Planning	Limited	Strong
Sample efficiency	Generally lower	Can be higher
Complexity	Lower	Higher

Analysis:

For highly dynamic environments, model-free methods can be robust when the environment is difficult to model, whereas model-based methods can be advantageous when reliable environment predictions are available.

Question 9 – Q-Learning Update

Q-learning is an off-policy, model-free reinforcement learning algorithm that learns the optimal action-value function.

The Q-learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- $Q(s,a)$ = current Q-value
- α = learning rate
- r = immediate reward
- γ = discount factor
- s' = next state
- $\max_{a'} Q(s',a')$ = best future action value

Example

Suppose:

$$Q(s,a)=5 \quad r=10 \quad \gamma=0.9 \quad \max_{a'} Q(s',a')=8$$

and:

$$\alpha=0.5$$

Then:

$$Q(s,a)=5+0.5[10+(0.9)(8)-5] = 5+0.5[10+7.2-5] = 5+0.5(12.2) = 11.1$$

Therefore, the updated Q-value becomes:

$$\boxed{Q(s,a)=11.1}$$

Repeated updates gradually improve the Q-values. As the agent gains experience, the values generally approach the optimal action values under appropriate learning conditions.

Analysis:

Q-learning converges toward optimal action values when the agent sufficiently explores the state-action space and appropriate learning conditions are satisfied.

Question 10 – On-Policy vs Off-Policy Learning

RL algorithms can be classified as **on-policy** or **off-policy** depending on how they learn from actions.

On-Policy Learning

An on-policy algorithm learns about the same policy that it uses to generate behavior.

Example: SARSA

SARSA updates using the action actually selected by the current policy.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

Off-Policy Learning

An off-policy algorithm learns an optimal target policy while potentially following a different behavior policy.

Example: Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Comparison

Feature	On-Policy	Off-Policy
Learning policy	Current behavior policy	Target policy can differ
Example	SARSA	Q-learning
Exploration	Directly reflected in learning	Can learn optimal policy while exploring
Behavior	Often more conservative	Can be more aggressive

Application	Useful where behavior safety matters	Useful for learning optimal strategies
-------------	--------------------------------------	--

Analysis:

On-policy learning considers the policy actually followed, while off-policy learning separates the behavior policy from the target policy. The appropriate method depends on the application's safety, exploration, and learning requirements.